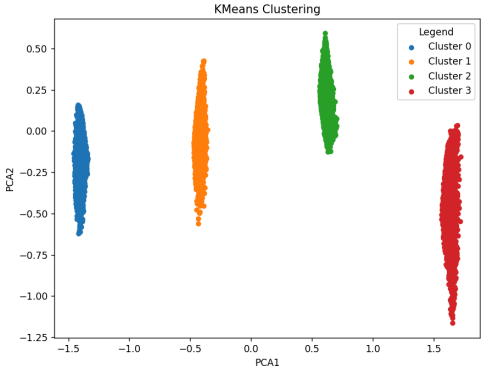
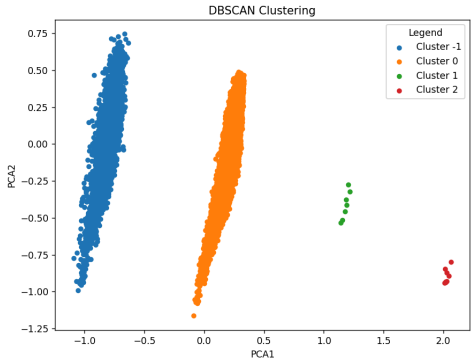
Music Popularity Analysis: Summary, Results, Insights, Challenges

Ananga Bajgai & Ewan Brinkman

Data Preprocessing: building from the EDA part of the project, we preprocessed our data by normalizing various numerical columns to be between 0 and 1. Additionally, the initial target variable of “track_popularity”, being numbers of range 0-100, was converted to simple categorical values of “low”, and “high”. This was to simplify the classification process, and since our 18,000 length dataset won’t be large enough to capture the complex patterns for 100 different class labels (thus, 2 class labels works better).

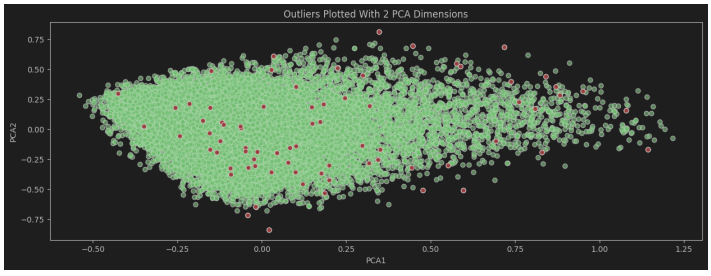
Cluster Analysis: two clustering algorithms were employed: Kmeans and DBScan. Since our data set was mainly numerical, with values between 0 and 1, Kmeans is appropriate. At the same time, our EDA work showed lots of outliers on various features, which would be a good reason to also try out DBScan, as a way to compare. The plots:

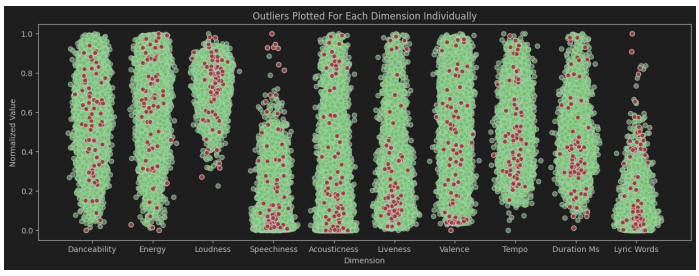




The first plot corresponds to the Kmeans, while the second corresponds to DBSCAN. The silhouette scores were 0.65 and 0.56, respectively. Despite trying various different parameters for DBSCAN, the resulting clusters were not great, with most datapoints being classified as noise/outlier (the blue color). On the other hand, Kmeans had a better silhouette score, and k=4, as shown in the plot above, had a very strong score.

Outlier Detection: we did Local Outlier Factor and Isolation Forest on all 10 features at once for outlier detection. For each algorithm, a plot was created using PCA dimensionality reduction to 2 features (left plot), and another plot where each of the 10 features was plotted (right plot). Red dots are outliers and green dots are inliers. The scale is 0 to 1 for the right plot since all the 10 features are numerical and normalized in the 0 to 1 range. A key observation is that there appears to be quite a lot of noise from the outlier detection. Here are the plots for local outlier factor, the isolation forest plots in the jupyter notebook are similar.





Another observation is for the plots on the right. All the outliers in the middle of the pack may not be an outlier for that specific feature, but may be an outlier because of another feature. For example, the red point at the very top of lyric words may be in the middle of danceability. Next, some of the points that are suspiciously far away from the clusters were manually verified. In the Jupyter Notebook, the Spotify song URL is pasted next to the suspicious outlier tracks, but all the songs flagged as outliers ended up being Ok and should stay in the dataset.

Feature Selection: the feature selection was done using Recursive Feature Elimination (RFE), and implemented using RFECV from sklearn, with 3-fold cross-validation. Since classification later is using SVM and Random Forest, RFE was done using estimators for both models. So for Random Forest, the optimal features were: 'loudness', 'speechiness', 'acousticness', 'liveness', 'valence', 'tempo', 'duration_ms', 'lyric_words'. For SVM, the optimal features were: 'danceability', 'energy', 'loudness', 'speechiness', 'acousticness', 'tempo', 'duration_ms', 'lyric_words'.

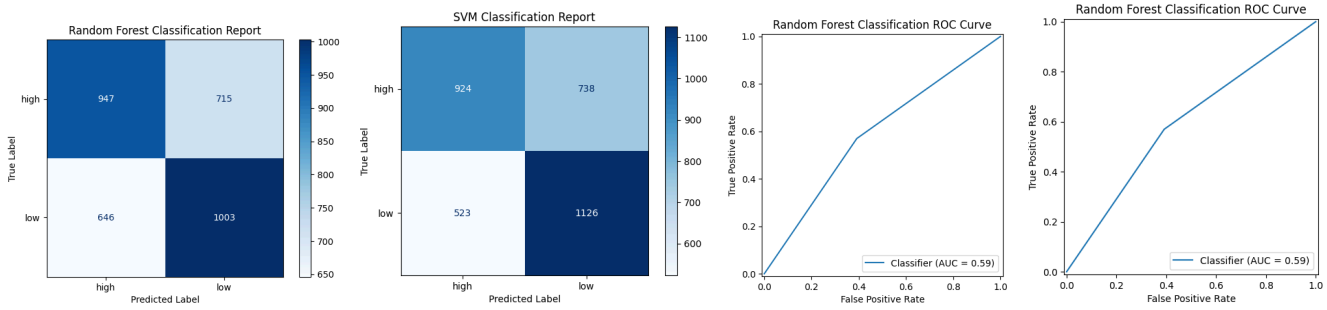
Overall, there was not a massive change between using and not using the optimal features, as shown by the result below. When not using the optimal columns, all the metrics of Random Forest and SVM had values of around 0.59-0.61 in terms of average 5-fold Cross-Validation score and in the test set. Then, using the optimal columns, these values did not change much

overall, with some decreasing down to .58, while others increasing to 0.62. However, we continue using the optimal columns to reduce the dimensionality.

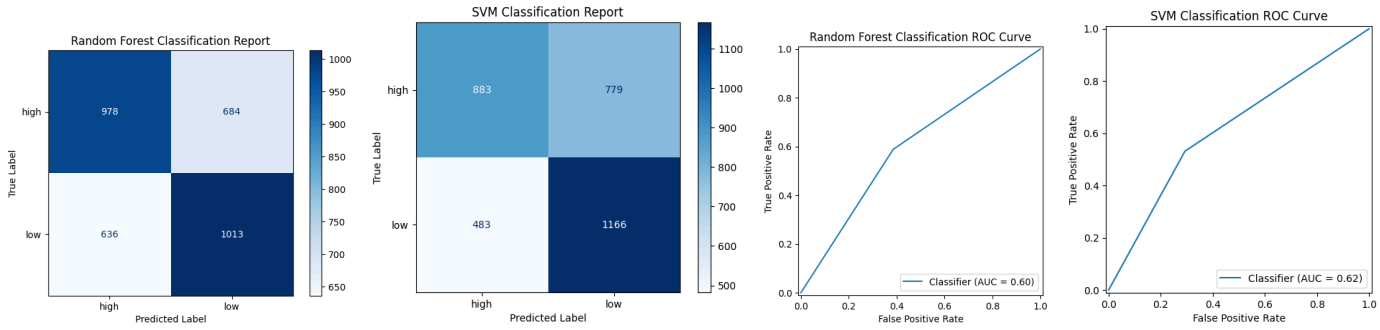
Classification and Hyperparameter: for classification, SVM and Random Forest Classifier were used. The full dataset was split into training and testing sets using a 80% training and 20% testing split, and 5-fold cross-validation was employed on the training set. As in the feature selection section, metrics of Accuracy, Precision, Recall, F1-score, and AUC-ROC score were used throughout in evaluating the models. Additionally, Random Search was used for each classifier for hyperparameter tuning. We compare the performance of the model before and after tuning on the test set, using the prior mentioned metrics and also their classification reports.

Comparing between hyperparameter tuning and and no tuning, there was a very slight improvement overall. For the 5-fold cross validation values, hyperparameter tuning increased most of the metrics by around 0.01 to 0.03. Similar trends held for the metrics of the test set as well, and this is shown in detail below with classification report and the ROC curve comparison between, before and after hyperparameter tuning. We do notice that SVM did not improve after hyperparameter tuning in terms of classifying the "high" values, but all other classifications improved after tuning in both the Classification report and ROC Curve.

Before hyperparameter tuning:



After hyperparameter tuning:



Key results and findings: Overall, there wasn't really a clear better model between the two, with both performing fairly evenly in all our metrics and plots. SVM seems to be slightly better at classifying "low" values and in the AUC metric, while worse than Random Forest for "high" values. In terms of Accuracy score, SVM had 62%, while Random Forest had 60%. Also, this shows that it is very hard to predict if a song will do well. This shows why it is hard for artists to continue making hit songs!

Domain-specific insights derived from the analysis: Our results show that it is hard to predict music popularity scores in the real-world. If it was easy to do so, then artists would be able to always have popular song hits. However, real world examples such as Taylor Swift's recent less popular album provide an example of how even the most popular artists may not always be able to always produce hit songs.

Challenges encountered and how they were addressed: A challenge was in converting the numerical "track_popularity" to categorical variables. Initially, we had more than two classes, representing the categorical version of "track_popularity", but these efforts suffered from poor results where the model would almost always select the middle range option, and rarely select more extreme ratings. Upon further investigation, it seemed likely due to class imbalance in the input dataset, and even efforts to balance the class imbalance, did not improve the results much. We were only able to improve the overall results more by having two class labels of "low" and "high". Additionally, we are also restricted by our data set, as having a larger dataset of more songs, may be able to capture the real-world complexity of song popularity perception better. Our data set only has 18,000 songs, which is quite small compared to the over 100 millions of songs just on Spotify, for instance. For example, most of the songs from the popular band, The Cure, are missing from the dataset.